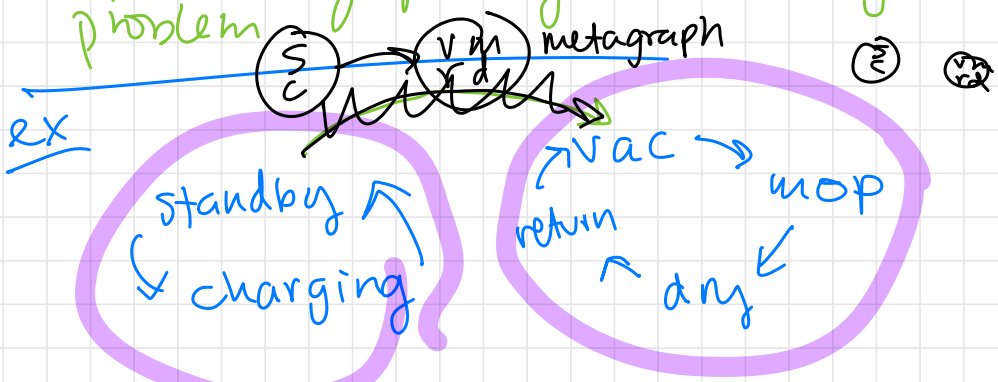
A robot can operate in one of n modes. You are given a list of m commands that change the robot from one mode to another; the reverse command may not exist. The manufacturer claims: "No matter which commands you execute, you can always restore the robot to the mode it was in before you started." Give an efficient algorithm to check this claim. (Note: there is an example input at the bottom of the page, but you should answer these questions for a general input.)

Command	Starting mode	Resulting mode
Begin charging	Standby	Charging
Stop charging	Charging	Standby
Switch to mop	Vacuuming	Mopping
Begin drying	Mopping	Drying
Return to base	Drying	Returning
Resume cleaning	Returning	Vacuuming

G

- what graph to build? node for every mode, edge for command
- Given that graph, what is relationship btwn your problem and graph? does metagraph have edges?
- Give the graph alg. to solve your problem



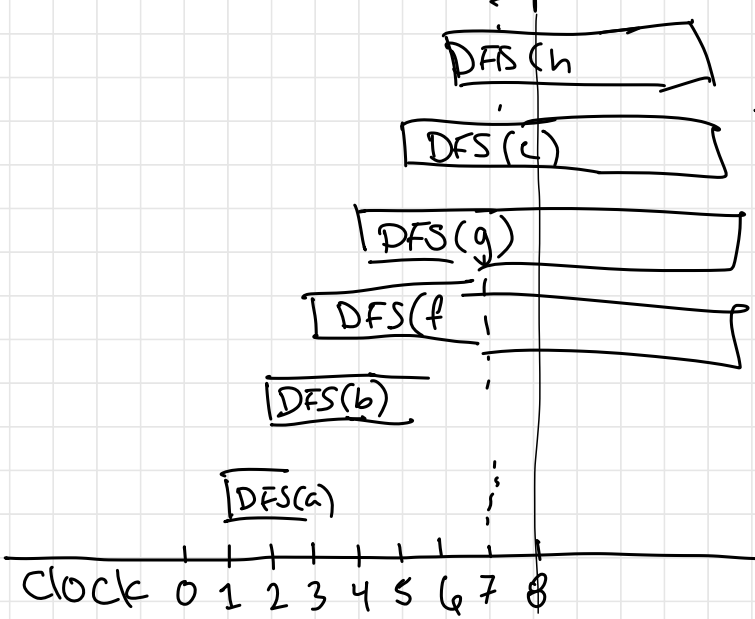
Strongly connected component SCCs
for all u,v in SCC, u→v and v→u

for all(v), set
v.pre = 0
v.post = 0
clock = 0

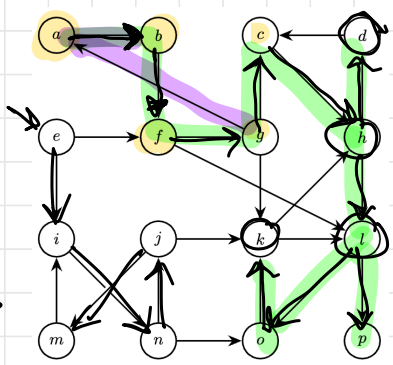
```
DFSALL(G):
    PREPROCESS(G)
    for all vertices v
        unmark v
    for all vertices v
        if v is unmarked
            DFS(v)
```

```
DFS(v):
    mark v                          clock++
    PREVISIT(v)                     v.pre = clock
    for each edge vw
        if w is unmarked
            parent(w) ← v
            DFS(w)
    POSTVISIT(v)                    clock++
                                    v.post = clock
```

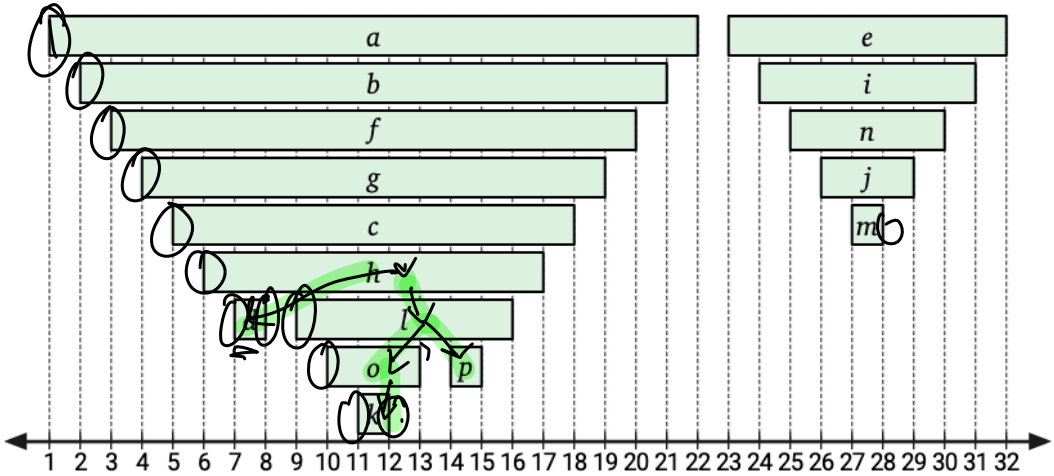
"clock": increment anytime we start or end a vertex's DFS call



DFS(a)
b.parent=a



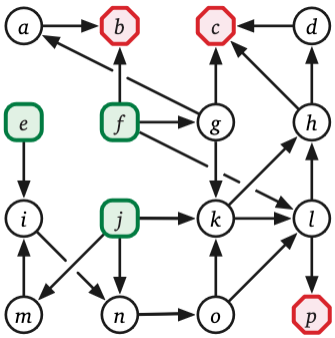
a.pre = 1
b.pre = 2
f.pre = 3
⋮
d.pre = 7
d.post = 8

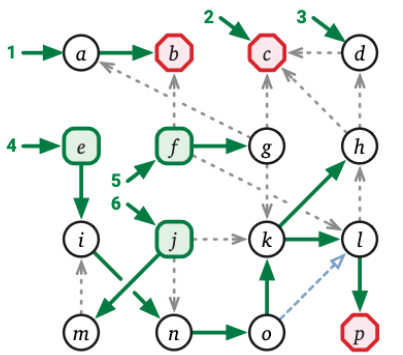


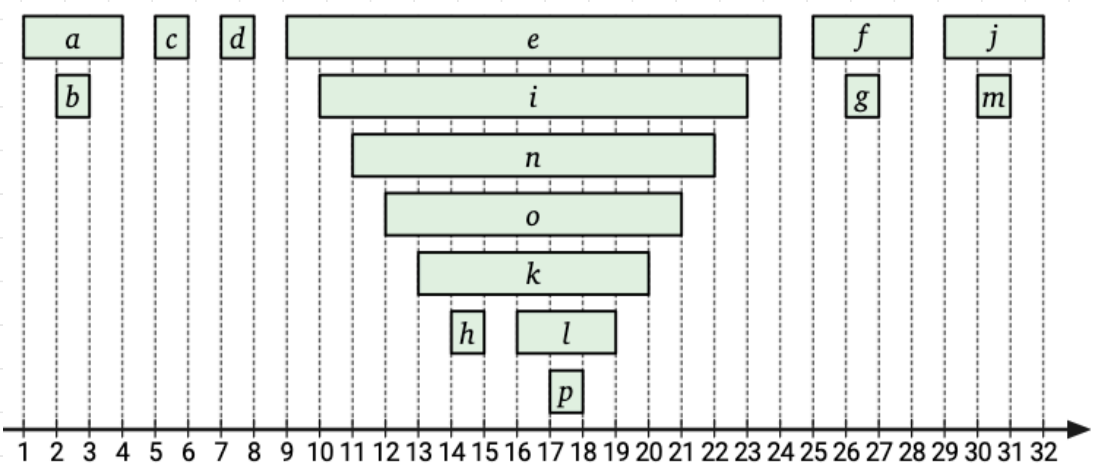
Pre: a b f g c h d l o k p e i n j m

Post: d k o p l h c g f b a m j n i e

[G has a cycle iff for some edge u→v,
u.post < v.post]


a
b
c
d
e
f
g
h
i
j
k
l
m
n
o
p


1
2
3
4
5
6
a
b
c
d
e
f
g
h
i
j
k
l
m
n
o
p


a
c
d
e
f
j
b
i
g
m
n
o
k
h
l
p
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31 32